

67101

מבחן במבוא למדעי המחשב – האוניברסיטה העברית  
פרופ' נעם ניסן  
מועד א' – 13.2.02  
זמן המבחן: שתיים וחצי

יש לענות על כל השאלות.

**שאלה 1.** (חלק אמריקאי, 40 נקודות – 5 נקודות לכל סעיף)

יש להקיף את התשובה הנכונה על גבי טופס השאלון, ולהגיש את השאלון המלא יחד עם מחברת הבחינה. אין לענות על שאלה זו במחברת הבחינה.

שתי השאלות הבאות משתמשות במחלקה זו:

```
class MyInt {  
    private int v;  
    public MyInt(int v) { this.v = v; }  
    public void set(int v) { this.v = v; }  
    public int get() { return v; }  
    public static void doIt1(MyInt x, int v) { x.set(v); }  
    public static void doIt2(int x, int v) { x=v; }  
}
```

1. מה ידפיס קטע הקוד הבא:

```
MyInt a = new MyInt(7);  
MyInt b = a;  
MyInt c = new MyInt(7);  
b.set(8);  
System.out.println("“ + a.get() + c.get());
```

- 87 .a
- 78 .b
- 77 .c
- 88 .d

2. מה ידפיס קטע הקוד הבא:

```
MyInt a = new MyInt(7);  
int b = 7;  
MyInt.doIt1(a, 8);  
MyInt.doIt2(b, 8);  
System.out.println("“ + a.get() + b);
```

- 78 .a
- 88 .b
- 77 .c
- 87 .d

3. אלו מהמאורעות הבאים קורים במסגרת הפעולות המתבצעות בתהליך שבסופו `XYZEvent` מטופל ע"י אובייקט מסוג `MyXYZListener` (במודל ה-Events של GUI)

- a. אובייקט מטיפוס `MyXYZListener` מועבר למתודה `addXYZListener` של רכיב גרפי מסוים
- b. אובייקט מטיפוס `XYZEvent` נוצר ע"י אובייקט מטיפוס `MyXYZListener`
- c. אובייקט מטיפוס `XYZEvent` קורא למתודה `addXYZListener` של רכיב גרפי מסוים
- d. אובייקט מטיפוס `MyXYZListener` נוצר ע"י אובייקט מטיפוס `XYZEvent`

4. מה מוחזר ע"י  $f(123,456)$

```
public static int f(int x, int y) {
    if (x == 0) return y;
    if (y == 0) return x;
    return f(x-1, y-1);
}
```

- a. 123
- b. 333
- c. 456
- d. 579

5. מה זמן הריצה הדרוש בכדי להכפיל שני מספרים טבעיים שכל אחד מהם באורך  $N$  ביטים (ההכפלה בשיטת "בית-ספר"; כל אחד מהמספרים נשמר במערך בוליאני)

- a.  $O(N \log N)$
- b.  $O(N^2)$
- c.  $O(N)$
- d.  $O(2^N)$

6. במתודה מסוימת מופיעה השורה הבאה:

```
Xxx x = new Xxx();
```

(כאשר `Xxx` הינה מחלקה שהוגדרה במקום אחר) אלו מהמשפטים הבאים נכון:

- a. ב-`Stack` מוקצה מקום למשתנה `x` ולאובייקט ש-`x` מצביע עליו
- b. ב-`Heap` מוקצה מקום למשתנה `x` ולאובייקט ש-`x` מצביע עליו
- c. ב-`Stack` מוקצה מקום למשתנה `x` וב-`Heap` לאובייקט ש-`x` מצביע עליו
- d. ב-`Heap` מוקצה מקום למשתנה `x` וב-`Stack` לאובייקט ש-`x` מצביע עליו

## שאלה 2. (30 נקודות)

בשאלה זו לא נדרש כל תיעוד.

א. (10 נקודות) כתבו מחלקה בשם `Time` המתארת נקודת זמן ביממה בדיוק של דקות. המחלקה תספק את השירותים הבאים:

a. `Time(int hours, int minutes)` – מייצר "זמן" חדש הנתון ע"י השעה והדקה הנתונים. על השעה להיות בתחום 0-23 ועל הדקה בתחום 0-59. יש לוודא שהקלט מתאים לדרישה זו, (ואחרת יש להטיל `IllegalArgumentException`).

b. `int getHours()` – מחזיר את השעה בזמן זה.

c. `int getMinutes()` – מחזיר את הדקה בזמן זה.

d. `boolean after(Time other)` – מחזיר `True` אם זמן זה (של האובייקט עליו מופעלת המתודה) מאוחר יותר ביממה מהזמן האחר (הנתון כפרמטר).

ב. (20 נקודות) כתבו מחלקה בשם `Schedule` המתארת לוח זמנים של נסיעות רכבת. לוח זמנים הינו אוסף זמני היציאה של הרכבת. המחלקה תספק את השירותים הבאים:

a. `Schedule(int maxNumTrains)` – מייצר לוח רכבות חדש בו ניתן יהיה להכניס לכל היותר `maxNumTrains` זמני יציאה. הלוח מאותחל להיות בעל זמן יציאה יחיד בשעה 00:00.

b. `void addTime(Time t) throws ScheduleFullException` – מוסיף זמן יציאה נוסף ללוח הזמנים. במידה ולוח הזמנים כבר מלא תיזרק ה-`EXCEPTION` הנזכרת (יש להגדיר גם את `ScheduleFullException` כיאות).

c. `Time findTrain(Time t)` – מחזיר את זמן היציאה המתאים לאדם שהגיע לתחנה בזמן `t` – כלומר זמן היציאה המוקדם ביותר שבא אחרי זמן `t`. (אם ישנה רכבת היוצאת בדיוק בזמן `t`, לא ניתן לתפוס אותה. אם אין כל רכבת היוצאת באותו יום לאחר זמן `t`, יש לתפוס את הרכבת הראשונה של היום הבא – 00:00).

בשאלה זו נדרש שזמן הריצה של `findTrain` יהיה  $O(\log n)$  – כאשר  $n$  הינו מספר זמני היציאה בלוח הזמנים (10 נקודות יורדו על מימוש איטי יותר). אין כל חשיבות לזמן הריצה של `addTime`.

שתי השאלות הבאות מתייחסות לקטע הקוד הבא:

```
public class A {  
    public A() { System.out.println("Created A"); }  
    public void doIt() { System.out.println("Did A"); }  
}  
public class B extends A {  
    public B() { System.out.println("Created B"); }  
    public void doIt() { System.out.println("Did B"); }  
}
```

7. מה מדפיסה הפקודה:

```
A a = new B();
```

Created A .a

Created B .b

Created A .c  
Created B

Created B .d  
Created A

8. לאחר ש-a נוצר בשאלה הקודמת (ע"י `A a = new B();`), מה מדפיסה הפקודה:  
`a.doIt();`

Did A .a

Did B .b

Did B .c  
Did A

Did A .d  
Did B

### שאלה 3 (30 נקודות – כל סעיף 10 נקודות)

בשאלה זו יש לתת תיעוד API מלא לכל המחלקות והמתודות הציבוריות.

בשאלה זו יש להגדיר מחלקה אבסטרקטית בשם Scene ושתי תתי-מחלקות קונקרטיות שלה: WallpaperScene ו-BlurredScene.

1. Scene הינה מחלקה אבסטרקטית המתארת תמונת רקע בגודל לא מוגבל – לכל נקודה במישור יש את גוון האפור שלה – מספר ממשי בין 0 ל 1.

a. `double colorAt(int x, int y)` – זוהי מתודה אבסטרקטית המחזירה את הצבע (גוון האפור)

של הנקודה  $(x,y)$  בתמונה. (אין כל הגבלה על הקואורדינטות  $(x,y)$ )

b. `Scene blur(int k)` – מתודה זו מחזירה גרסה מטושטשת  $k$ -פעמים של תמונה זו. המתודה

משתמשת במחלקה BlurredScene – ראו בסעיף 3 לעיל. לדוגמא, אם `pic` הינו אובייקט מטיפוס Scene אזי `pic.blur(2)` יחזיר את

`new BlurredScene(new BlurredScene(pic))`

`pic.blur(0)` יחזיר את התמונה המקורית, `pic`. אם `k` שלילי יש להחזיר, גם כן, את התמונה

המקורית. מתודה זו ממומשת במלואה במחלקה זו ואינה נדרסת (OVERRIDDEN) בתתי-המחלקות.

2. WallpaperScene מתארת תמונה הנוצרת ע"י ריצוף של מטריצה נתונה, כלומר חזרה אינסופית של המטריצה הסופית. לדוגמא, אם המטריצה היא בגודל  $10 \times 100$  אזי הצבע בנקודה  $(456,789)$  של ה-WallpaperScene יהיה הצבע בנקודה  $(6,89)$  של המטריצה.

a. `WallpaperScene(double[][] matrix)` – מייצר תמונת רקע אינסופית המתקבלת ע"י

ריצוף של המטריצה `matrix`. ניתן להניח ש `matrix` הינה אכן מטריצה של גווני אפור,

כלומר שכל השורות בה באותו אורך, ושכל הכניסות בה הינן בתחום  $0-1$ . בשאלה זו ניתן

להניח שאפשר להשתמש במטריצה מבלי להעתיק אותה.

b. `double colorAt(int x, int y)` – מחזירה את הצבע בנקודה  $(x,y)$  בתמונה – הצבע הינו

הצבע של הכניסה במטריצה המתאימה לקואורדינטות  $(x,y)$  לפי הריצוף. לדוגמא, אם

המטריצה עמה נוצר ה-WallpaperScene היא בגודל  $10 \times 100$  אזי `colorAt(456,789)`

יחזיר את הכניסה ה- $(6,89)$  של המטריצה.

3. BlurredScene מתארת תמונה הנוצרת ע"י טשטוש של תמונה אחרת. הטשטוש מתקבל על ידי כך

שהצבע בנקודה מסוימת של התמונה המטושטשת הינו הממוצע של הצבעים של כל השכנים של הנקודה

בתמונה המקורית (כל תשע השכנים – כולל הנקודה עצמה).

a. `BlurredScene(Scene original)` – מייצר גרסה מטושטשת של התמונה הנתונה. ניתן להניח

שהתמונה המקורית הינה `immutable` ולכן ניתן להשתמש בה מבלי להעתיק אותה.

b. `double colorAt(int x, int y)` – מחזירה את הצבע בנקודה  $(x,y)$  בתמונה – הצבע הינו

הממוצע של תשע הנקודות  $(x-1,y-1), \dots, (x+1,y+1)$  בתמונה המקורית.

**בהצלחה!!**